

INFORME TÉCNICO

Base de conocimiento vectorial - CODEFEST AD ASTRA 2026

Etapa 1 | Diseño, evolución, problemas encontrados y decisiones finales

Objetivo: Construir una base vectorial reproducible que recupere los 3 documentos y 10 fragmentos más relevantes para cada una de las 50 consultas, manteniendo trazabilidad con los DOC_ID oficiales y sin emplear modelos generativos durante la recuperación.

Resumen ejecutivo

La solución final procesa el corpus multiformato oficial, conserva la identidad documental definida por Indice_Datos_Codefest.xlsx, fragmenta el contenido en unidades lingüística o estructuralmente completas, genera embeddings densos con BAAI/bge-m3 y los indexa en FAISS mediante IndexFlatIP. La recuperación combina similitud semántica con un reranking determinista basado en coincidencias léxicas, equivalencias de dominio, pares conceptuales y una penalización ligera de ruido bibliográfico. No se implementó grafo de conocimiento.

Indicador	Resultado final
Documentos oficiales	1.826
Documentos con chunks	1.820
Chunks / vectores	185.376
Encoder final	BAAI/bge-m3
Dimensión	1.024
FAISS	IndexFlatIP + normalización L2
Salida	50 consultas; 3 docs + 10 fragmentos
Grafo de conocimiento	No implementado (opcional)

Arquitectura final

1	2	3	4	5	6	7	8
Corpus ADL	IDs oficiales	Extracción y limpieza	Chunking seguro	BGE-M3 1024D	L2 + IndexFlatIP	Reranking determinista	resultados .jsonl

La evaluación del reto exige un PDF técnico de máximo 8 páginas y reproducibilidad mediante generador.py. Este documento está estructurado para funcionar también como memoria de decisiones del proyecto.

1. Estrategia de chunking

Se adoptó una estrategia híbrida multiformato, guiada por dos restricciones: conservar unidades completas y no superar 250 palabras por fragmento. En prosa se priorizan límites de oración; en fuentes estructuradas se usan límites naturales del registro.

Tipo de fuente	Unidad de segmentación	Criterio
PDF / TXT / texto OCR	Oraciones y bloques completos	No cortar una oración; reconocer abreviaturas, iniciales y signos Unicode.
JSON	Campos, párrafos, listas y registros	Mantener contexto clave:valor y orden del contenido.
CSV / XLSX	Fila/campo	Preservar el nombre de columna como contexto.
PBF	Feature y atributos	Representar atributos como pares; evitar duplicación estructural.
JPG / AVIF con texto	Texto extraído/OCR	Solo se indexa texto verificable; no se inventa contenido.

1.1 Tamaño y completitud

Objetivo de empaquetado final: aproximadamente 700 tokens y 200 palabras.

Límites finales: máximo 8.192 tokens del encoder y máximo 250 palabras del reto.

Una unidad semántica o estructural nunca se corta solo para alcanzar un tamaño fijo.

El segmentador reconoce puntos, interrogaciones y exclamaciones occidentales y Unicode (incluido chino/japonés/árabe), además de cierres de comillas y paréntesis.

Los bloques que parecen listas o tablas se tratan como datos estructurados, evitando interpretar sus celdas como prosa.

1.2 Metadata y trazabilidad

Se conservaron únicamente los campos obligatorios para reducir ambigüedad y tamaño del almacén. Cada línea de metadata.jsonl corresponde exactamente al vector insertado en la misma posición de FAISS.

Campo	Función
doc_id	Identidad oficial del documento.
chunk_id	Identidad única del fragmento.
fuelle	Ruta del archivo original.
formato	Extensión real del origen.
fenomeno	1, 2 o 3; usado también como post-filtro.
posicion	Orden del chunk dentro del documento, desde 0.
num_tokens	Conteo con el tokenizador BGE-M3.
texto	Contenido original recuperable.

1.3 Cierre de casos especiales

Tres JSON de Alertas Tempranas quedaron pendientes porque body_paragraphs contenía largas listas de códigos NNN-NN, que no son oraciones. Se trataron los códigos como elementos atómicos y se agruparon sin alterar su orden. Esto produjo 6 chunks nuevos y cerró el corpus en 185.376 chunks válidos.

2. Encoder seleccionado y embeddings

2.1 Evolución: multilingual-e5-base -> BAAI/bge-m3

La primera opción fue intfloat/multilingual-e5-base. Se escogió inicialmente por su soporte multilingüe y buen desempeño en recuperación, pero su límite real de 512 tokens entró en conflicto con el requisito de no cortar oraciones. El corpus contiene oraciones completas en chino, japonés y árabe que exceden 512 tokens sin superar 250 palabras.

Versión	Límite usado	Resultado / problema
E5 v1	450 tokens internos	823 bloqueantes; 450 era un margen arbitrario, no el límite real.
E5 v2	512 tokens	623 no resueltos; persistían oraciones completas demasiado largas.
BGE-M3 final	8.192 tokens	Permite mantener completitud lingüística. Máximo observado del corpus final: 897 tokens.

Decisión final: BAAI/bge-m3 se eligió porque combina capacidad multilingüe, embeddings densos de 1.024 dimensiones y una ventana suficientemente amplia para respetar las fronteras lingüísticas. En la indexación se utilizó max_length=1024 porque el mayor chunk final tenía 897 tokens; el modelo ofrece margen adicional para el diseño del chunking.

2.2 Codificación final

Implementación: BGEM3FlagModel de FlagEmbedding; solo dense embeddings (sin sparse ni ColBERT en el índice entregado).

Construcción en GPU Tesla T4 con use_fp16=True; embeddings convertidos a float32 antes de FAISS.

Normalización L2 explícita de cada vector. Las normas mínima, máxima y media validadas fueron 1.000000.

185.376 vectores de dimensión 1.024; la creación en Colab tomó 00:54:19.

3. Tipo de FAISS

Se empleó faiss.IndexFlatIP(1024). IndexFlatIP realiza búsqueda exacta por producto interno; al normalizar consulta y documentos a norma unitaria, el producto interno equivale a similitud coseno. Se prefirió exactitud sobre índices aproximados porque el tamaño del corpus era manejable y la evaluación premia el orden fino de relevancia.

Propiedad	Valor
Índice	IndexFlatIP
Métrica efectiva	Coseno (Inner Product sobre L2-normalizados)
Vectores	185.376
Dimensión	1.024
index.faiss	724,13 MB
metadata.jsonl	258,26 MB

4. Identidad documental, extracción y trazabilidad

4.1 Problema crítico de DOC_ID

En una etapa temprana se generaron identificadores secuenciales propios del tipo DOC-000xxx. Posteriormente, las FAQ aclararon que la evaluación a nivel de documento usaría el DOC_ID oficial de Indice_Datos_Codefest.xlsx. Mantener IDs inventados habría invalidado el emparejamiento F1@3 aunque el texto recuperado fuera correcto.

Se reconstruyó el corpus tomando el Excel oficial como única autoridad de identidad.

Se reemplazaron todos los identificadores inventados por IDs como F1-AIINDEX-021, F2-UNOOSA-030 o F3-ALERTAS-xxx.

El emparejamiento se realizó por ruta relativa completa (Carpeta + Nombre estandarizado), no solo por nombre de archivo, porque existían nombres duplicados.

Se excluyeron archivos locales que no pertenecían al inventario y se conservaron los 1.826 documentos oficiales.

4.2 Extracción multiformato

El pipeline inicial estaba centrado en PDF; se amplió a todos los formatos del corpus. Para datos estructurados se preservó el contexto de los campos; para PBF se convirtieron atributos cartográficos a texto; para imágenes se aplicó OCR solo cuando existía texto verificable.

4.3 Extracción PDF corrupta

F3-CEOBS-030: La extracción directa devolvía texto con mapeo de fuente roto (por ejemplo, cadenas semejantes a 0LQDPDWD&RQYHQWLRQ...). Se forzó OCR sobre el PDF original y se sustituyó únicamente ese contenido defectuoso. La decisión evitó indexar ruido sin alterar el resto de documentos.

4.4 Metadata final

La validación final comprobó 1.826 documentos oficiales, 1.820 con chunks y 6 sin contenido textual indexable. Se verificó: 0 DOC_ID no oficiales, 0 chunk_id duplicados, posiciones consecutivas, fuente/formato/fenómeno consistentes, num_tokens recalculado con BGE-M3 y 0 violaciones de 250 palabras.

4.5 Correspondencia FAISS <-> metadata

FAISS usa IDs enteros internos. No se exigió que ese entero fuera igual al texto de chunk_id: la correspondencia se garantizó insertando los vectores y escribiendo metadata.jsonl en exactamente el mismo orden. Al recuperar un ID interno i, metadata[i] devuelve el chunk correcto.

5. Problemas e inconvenientes encontrados

Problema	Diagnóstico	Cambio aplicado / por qué
IDs propios	No coincidían con DOC_ID oficial.	Reconstrucción contra el Excel oficial; evita fallos de evaluación documental.
Pipeline PDF-only	El corpus incluía JSON, CSV, XLSX, PBF e imágenes.	Extracción y chunking multiformato; evita perder más de una modalidad del corpus.
450 tokens en E5	Margen interno tratado como límite duro.	Se corrigió a 512; luego se descartó E5 al persistir bloqueantes.
623 unidades >512	Oraciones multilingües completas no podían cortarse.	Cambio a BGE-M3; preserva completitud lingüística.
PDF con fuente corrupta	Texto ilegible aunque el PDF era visualmente válido.	OCR forzado solo para F3-CEOBS-030.
3 JSON residuales	Listas de códigos se confundían con prosa.	Códigos tratados como elementos estructurados atómicos.
metadata Colab dañada	JSONDecodeError después de >150k líneas.	Re-subida y verificación de línea total/hash; nunca se saltó una línea.
index.faiss local truncado	faiss.read_index() devolvió read error.	Comparación de tamaño/SHA-256 y nueva copia; no se regeneraron embeddings.
Ranking semántico demasiado general	q001 priorizaba IA/seguridad general sobre CBRN.	Reranking determinista con términos equivalentes y conceptos críticos.

5.1 Principio usado para resolver problemas

Cada corrección buscó conservar información y trazabilidad antes que “forzar” el proceso. Cuando una unidad no podía dividirse con seguridad se marcaba como pendiente; cuando un archivo estaba corrupto se reextraía; cuando un artefacto transferido fallaba se verificaba integridad. Esto evitó silenciosamente truncar texto, omitir documentos o desalinear vectores y metadata.

5.2 Controles de integridad operativos

JSONL: parseo de cada línea y conteo esperado antes de embeddings.

FAISS: read-back del índice, ntotal=185.376 y d=1.024.

Vectores: finitud, norma no nula y normalización L2.

Transferencias: tamaño de archivo y SHA-256 cuando hubo corrupción.

Salida: 50 líneas, q001-q050, exactamente 3 documentos y 10 fragmentos, <=250 palabras.

6. Evolución y ajuste de generador.py

El generador se afinó usando las 50 consultas públicas y revisión cualitativa de relevancia; no existía ground truth público. Los cambios no fijan DOC_ID concretos ni respuestas. Se aplican reglas uniformes sobre texto, puntuaciones y metadata.

Etapa	Comportamiento	Aprendizaje
Baseline dense	BGE-M3 + FAISS; top por similitud.	Buena semántica general, pero algunos términos técnicos quedaban detrás de resultados amplios.
7.1 híbrida	85% score normalizado + 15% léxico.	El min-max local daba una ventaja excesiva al primer dense; mejora insuficiente.
7.2	Score dense crudo + 0,18 léxico + 0,10 crítico.	q001 movió CBRN/CBRNE a los primeros lugares; q017 siguió funcionando por semántica y q033 mejoró con léxico.
Final	Pool mayor + pares conceptuales + penalización de ruido.	Mejor cobertura de consultas compuestas y reducción de bibliografías/URLs como falsos positivos.

6.1 Fórmula final de reranking

score_final: $\text{score_dense} + 0,18 \cdot \text{score_lexico} + 0,10 \cdot \text{score_critico} + 0,12 \cdot \text{score_pareja} - 0,05 \cdot \text{score_ruido}$

score_dense: similitud de BGE-M3 devuelta por FAISS.

score_lexico: cobertura literal y de alias multilingües/de dominio.

score_critico: refuerzo de conceptos muy discriminativos (NBQR/CBRN, ASAT, RPO, spoofing, minería ilegal, etc.).

score_pareja: exige coocurrencia cuando la pregunta combina dos ideas, por ejemplo rutas aéreas + tráfico, petróleo + grupos armados, crimen organizado + instituciones.

score_ruido: penaliza ligeramente fragmentos dominados por URLs, referencias o indicadores bibliográficos.

6.2 Recall, filtros y agregación documental

TOP_FAISS=10.000 y hasta 1.000 candidatos del fenómeno correspondiente antes del reranking.

Filtro de fenómeno: q001-q016 -> F1; q017-q032 -> F2; q033-q050 -> F3.

Documentos: mejor score de chunk + 0,05 del segundo mejor chunk del mismo documento.

Fragmentos: primeros 10 del ranking final, todos pre-validados <=250 palabras.

No se usa decoder, resumen, expansión generativa de consulta ni selección mediante LLM.

7. Conceptos y terminología básica

Término	Definición en este proyecto
Documento	Archivo original del corpus identificado por DOC_ID.
Chunk / fragmento	Unidad de texto que se codifica e indexa individualmente.
Token	Unidad que utiliza el tokenizador del modelo; no equivale necesariamente a una palabra.
Tokenizer	Componente que convierte texto en tokens antes de pasar al encoder.
Encoder	Modelo que transforma texto en una representación vectorial; no genera la respuesta.
Decoder / LLM generativo	Modelo que genera texto autoregresivamente; no se usa en la recuperación final.
Embedding	Vector numérico que representa semánticamente un fragmento o consulta.
Dimensión	Cantidad de valores del embedding; BGE-M3 produce 1.024 en este pipeline.
Normalización L2	Escalado del vector para que su norma sea 1.
Similitud coseno	Medida angular entre vectores; con L2 se obtiene mediante producto interno.
FAISS	Biblioteca para almacenar y buscar vectores eficientemente.
IndexFlatIP	Índice FAISS exacto por Inner Product, sin aproximación.
Metadata	Información externa asociada a cada vector: IDs, fuente, fenómeno, texto, etc.
JSONL	Formato donde cada línea es un objeto JSON independiente.
Top-k	Los k resultados con mayor puntuación.
Reranking	Reordenamiento de candidatos después de la primera recuperación.
Post-filtro	Regla posterior basada en metadata o score, p. ej. filtrar por fenómeno.
Alias de dominio	Equivalencia controlada, p. ej. NBQR <-> CBRN/CBRNE o antisatélite <-> ASAT.
OCR	Reconocimiento óptico de caracteres para recuperar texto desde imágenes/PDF visual.
F1@3	Métrica de coincidencia de los 3 documentos recuperados respecto al ground truth.
NDCG@10	Métrica que evalúa relevancia y posición de los 10 fragmentos recuperados.
Grafo de conocimiento	Red de entidades y relaciones explícitas; componente opcional en este reto.

8. Grafo de conocimiento, validación y reproducibilidad

8.1 Grafo de conocimiento: no aplica en la entrega

No se implementó grafo de conocimiento. El componente era opcional y añadía una capa adicional de NER, extracción de relaciones, resolución de entidades y fusión con resultados vectoriales. Dado que el objetivo principal era asegurar una base exacta, reproducible y correctamente alineada con el inventario, se priorizó cerrar la recuperación vectorial antes de introducir una fuente adicional de ruido. Por lo tanto, no se entrega grafo.graphml.

Como evolución futura, el grafo podría conectar países, organizaciones, tecnologías, recursos, grupos armados, lugares y eventos; sus chunks candidatos podrían fusionarse con FAISS mediante RRF u otra estrategia no generativa.

8.2 Validación final

Control	Resultado
DOC_ID oficiales	1.826 inventario; 0 IDs no oficiales
Chunks	185.376; chunk_id únicos; posiciones válidas
Límites	Máximo 897 tokens observado; máximo 250 palabras permitido
FAISS	185.376 vectores; dimensión 1.024; read-back OK
Normas	mín.=máx.=media=1,000000
Resultados	50 líneas; q001-q050; 3 docs + 10 fragments
Reproducibilidad	generador.py funciona con argumentos opcionales y valores por defecto

8.3 Estructura de entrega y ejecución

Ruta	Contenido
generador.py	Recuperación y generación de resultados.
requirements.txt	FlagEmbedding, faiss-cpu, numpy, torch.
resultados.jsonl	50 resultados finales.
base_vectorial/encoder_bge-m3/index.faiss	Índice vectorial serializado.
base_vectorial/encoder_bge-m3/metadata.jsonl	Metadata alineada con los IDs internos de FAISS.
informe_tecnico.pdf	Este documento técnico.
Comando reproducible: python generador.py --consultas consultas.jsonl --base-vectorial ./base_vectorial --salida resultados.jsonl	

Conclusión

La solución final favorece integridad, completitud lingüística y trazabilidad. El cambio de E5 a BGE-M3 resolvió el principal conflicto técnico entre ventana de contexto y oraciones multilingües largas; IndexFlatIP evitó error aproximado; y el reranking determinista mejoró términos técnicos sin introducir modelos generativos. El resultado es una base vectorial reproducible y compatible con el contrato del reto.

Fuentes de trazabilidad del proyecto

[E1] Reto clasificatorio CODEFEST AD ASTRA 2026 / CODEFEST_2026-1.pdf. [FAQ] Preguntas Frecuentes.xlsx. [IDX] Indice_Datos_Codefest.xlsx. [P4] paso_4_final_bge_m3.py y resolver_3_json_pendientes_bge_m3.py. [P5] paso_5_generar_metadata_bge_m3.py. [P6] paso_6_crear_embeddings_faiss_bge_m3.py. [GEN] generador.py final.